

Objetivo del programa

El **Certificado Microsoft Python Developer Professional** de Coursera busca formar a desarrolladores intermedios de Python, brindándoles las habilidades necesarias para crear aplicaciones innovadoras en ámbitos como:

- Desarrollo web
- Ciencia de datos y machine learning
- Automatización de tareas
- Juegos y aplicaciones móviles
- Informática científica

Cursos del programa (6 en total)

1. **Fundamentos de Programación en Python**
2. **Análisis de Datos y Visualización con Python**
3. **Automatización y Scripting con Python**
4. **Desarrollo Web con Python**
5. **Técnicas Avanzadas de Desarrollo con Python**
6. **Desarrollo de Proyectos en Python**

Contenido del curso inicial: *Fundamentos de Programación en Python*

Este curso se divide en **6 módulos**:

1. **Introducción a Python**
 - Historia, características y aplicaciones.
 - Configuración del entorno y primeros programas.

- Actividad: práctica de depuración básica.

2. Conceptos básicos de Python

- Variables, tipos de datos, operadores, control de flujo.
- Actividad: creación y manipulación de listas; uso de bucles.

3. Funciones y módulos

- Funciones, clases, objetos, módulos y su utilidad.
- Actividad: uso creativo de funciones integradas.

4. Estructuras de datos en Python

- Listas, tuplas, diccionarios y conjuntos.
- Proyectos:
 - Ejercicios prácticos de estructuras de datos.
 - Algoritmos de ordenación y búsqueda.

5. Manejo de errores y depuración

- Identificación de excepciones comunes.
- Uso de herramientas de depuración.
- Actividad: práctica con excepciones y debugging.

6. Pruebas y control de versiones

- Tipos de pruebas y fundamentos.
- Uso de pytest para funciones de prueba.
- Git para control de versiones y colaboración.
- Proyecto final: script con manejo de datos, errores y depuración.

Bienvenido al mundo de la programación

La **programación** es el arte de dar instrucciones claras a un ordenador para que realice tareas específicas, igual que una receta indica cómo preparar un platillo. Estas

instrucciones, llamadas **código**, permiten a la máquina ejecutar pasos de manera lógica y ordenada.

En un programa se pueden:

- Tomar decisiones con sentencias condicionales (*if, elif, else*).
- Repetir acciones mediante bucles (*for, while*).
- Almacenar y organizar información con **variables** y **estructuras de datos** (listas, diccionarios, etc.).

Los **lenguajes de programación** son la forma en que los humanos nos comunicamos con los ordenadores. Cada uno tiene su propia sintaxis y aplicaciones:

- **Java**: aplicaciones empresariales.
- **C++**: juegos y sistemas por su alto rendimiento.
- **JavaScript**: desarrollo web interactivo.
- **Python**: versátil, sencillo y accesible, ideal para principiantes.

En definitiva, **Python** es un excelente punto de partida para adentrarse en este mundo porque combina facilidad de aprendizaje con un amplio rango de aplicaciones, desde la ciencia de datos hasta el desarrollo web.

¿Por qué Python? La puerta al éxito en programación

1. Legibilidad y facilidad de uso

Python destaca por su sintaxis sencilla, cercana al inglés, lo que facilita el aprendizaje.

Ejemplo: Calcular el área de un rectángulo

- **Python:**

```
length = 10
width = 5
area = length * width
print("The area of the rectangle is:", area)
```

Salida:

The area of the rectangle is: 50

- **C++:**

```
#include <iostream>
int main() {
    int length = 10;
    int width = 5;
    int area = length * width;
    std::cout << "The area of the rectangle is: " << area <<
std::endl;
    return 0;
}
```

Salida:

The area of the rectangle is: 50

En comparación, Python es más corto y legible que C++.

2. Versatilidad

Python se puede usar en múltiples áreas gracias a sus **bibliotecas y frameworks**:

- **Análisis de datos:**
 - *Pandas*: manipulación de datos en tablas.
 - *NumPy*: cálculos numéricos y trabajo con matrices.
- **Desarrollo web:**
 - *Django*: framework completo para aplicaciones (web) grandes.
 - *Flask*: framework ligero y flexible.
- **Inteligencia artificial y machine learning:**
 - *Scikit-learn*: algoritmos de clasificación, regresión y clustering.

- *PyTorch*: modelos de aprendizaje profundo para visión e NLP.

3. Comunidad activa

Python cuenta con una comunidad amplia que aporta recursos, tutoriales y soporte en línea, lo que facilita tanto el aprendizaje como la resolución de problemas.

En resumen: Python combina **simplicidad, versatilidad y apoyo de la comunidad**, lo que lo convierte en un lenguaje ideal para principiantes y también para proyectos avanzados.

Escenarios del mundo real: Python en acción

Python se aplica en múltiples campos gracias a su versatilidad:

1. Redes sociales

- Un analista accede de forma ética a datos de una plataforma usando su **API**.
- Python facilita la conexión mediante bibliotecas para recuperar y analizar la información.

2. Finanzas

- Un analista financiero automatiza informes bursátiles con la librería **yfinance**.
- Permite descargar datos históricos y en tiempo real, calcular indicadores y generar reportes con tablas y gráficos.

3. Educación

- Un profesor crea un cuestionario interactivo con **Tkinter**.
- Se construyen interfaces gráficas con botones, etiquetas y ventanas para evaluar a los alumnos de manera dinámica.

Estos ejemplos muestran cómo Python impulsa soluciones prácticas en análisis de datos, automatización y desarrollo de aplicaciones interactivas.

Puntos de vista opuestos

- **Argumento en contra:** Algunos defienden aprender lenguajes de bajo nivel (C++, C#) primero, ya que ayudan a comprender mejor el hardware y la administración de memoria.
- **Respuesta:** Para principiantes, esto puede ser abrumador. Python, en cambio, permite enfocarse en la **lógica y resolución de problemas**, como aprender a conducir con transmisión automática antes que manual.

Conclusión

- Python es un **punto de partida ideal** gracias a su legibilidad, simplicidad y fuerte comunidad.
- Aprenderlo primero ofrece una **curva de aprendizaje más suave** y una experiencia motivadora.
- Una vez dominados los fundamentos, se pueden explorar otros lenguajes más complejos según los intereses profesionales.
- La **práctica constante** es clave para ganar confianza y destreza.

En definitiva: Python abre la puerta al mundo de la programación de forma accesible y poderosa, invitando a explorar infinitas posibilidades.

Fundamentos de Programación (Traducción)

1. Variables

- Contenedores para almacenar valores de datos.
- Cada variable tiene un nombre y puede guardar distintos tipos de información (números, texto, listas, etc.).
- Ejemplo en Python:

```
edad = 25
```

```
nombre = "Ana"
```

2. Tipos de datos

- Definen el tipo de información que se almacena en una variable.
- Tipos comunes:
 - **int** → números enteros (ej. 10, -3)
 - **float** → números decimales (ej. 3.14)
 - **str** → cadenas de texto (ej. "Hola")
 - **bool** → valores booleanos (True/False)
 - **list, tuple, dict, set** → estructuras de datos

3. Operadores

- Símbolos que realizan operaciones sobre variables y valores.
- Categorías:
 - **Aritméticos:** `+`, `-`, `*`, `/`, `%`, `**`
 - **Comparación:** `==`, `!=`, `<`, `>`, `<=`, `>=`
 - **Lógicos:** `and`, `or`, `not`
 - **Asignación:** `=`, `+=`, `-=`, etc.

4. Sentencias de control de flujo

- Determinan cómo se ejecutan las instrucciones de un programa.

- **Condicionales:**

```
if edad >= 18:
    print("Adulto")
else:
    print("Menor de edad")
```

- **Bucles:**

```
for i in range(5):
    print(i)    # imprime números del 0 al 4
```

5. Funciones

- Bloques de código reutilizable que realizan una tarea específica.
- Mejoran la legibilidad, la modularidad y evitan repetir código.
- Ejemplo:

```
def saludar(nombre):
    return f"Hola, {nombre}!"

print(saludar("Ana"))
```

Comenzar a trabajar con Python

En la sesión de hoy se buscó **identificar formas en que Python puede simplificar, automatizar o mejorar tareas** en la vida personal o profesional.

Enfoque de la sesión:

- Analizar rutinas diarias y tareas frecuentes.
- Reconocer actividades repetitivas o que consumen mucho tiempo.
- Explorar aplicaciones de Python en necesidades reales.

Fortalezas destacadas:

- Excelente capacidad para detectar patrones y tareas repetitivas, incluso en contextos no convencionales como videojuegos.
- Conexión clara entre conceptos de programación (ej. bucles) y soluciones prácticas (alarmas, gestión de gastos).

Áreas de mejora:

- Al inicio fue necesaria algo de guía para identificar tareas automatizables.
- Con la práctica, será más sencillo detectar oportunidades por cuenta propia.

Conclusión:

Has mostrado un gran potencial para aplicar Python de forma significativa en tu vida. Con cada avance, podrás descubrir y documentar nuevas oportunidades para aprovechar este lenguaje en tu desarrollo personal y profesional.

Características principales de Python

Lenguaje interpretado

- Python no necesita compilarse antes de ejecutarse, el código se lee e interpreta línea por línea.
- Esto facilita la detección de errores y permite probar fragmentos de código rápidamente.
- Ventaja: mayor flexibilidad y rapidez en el desarrollo.

Lenguaje de alto nivel

- Está diseñado para ser fácil de leer y escribir, usando una sintaxis cercana al lenguaje humano.
- Permite centrarse en la **lógica de programación** en lugar de los detalles complejos del hardware o la administración de memoria.
- Ideal para principiantes y también eficiente para proyectos complejos.

Programación Orientada a Objetos (POO)

- Python soporta el paradigma de **Programación Orientada a Objetos**, lo que significa que organiza el código en **clases y objetos**.
- Esto fomenta la reutilización, modularidad y mantenimiento del código.

El poder de Python

Legibilidad y simplicidad

- Python es popular porque su sintaxis es **limpia, sencilla y parecida al inglés**.
- Permite enfocarse en la **resolución de problemas** y no en la sintaxis complicada.
- Aunque es amigable para principiantes, **no es un lenguaje de juguete**: se usa en desarrollo web, ciencia de datos, aprendizaje automático y más.

La belleza de la sintaxis de Python

- A diferencia de otros lenguajes llenos de símbolos o llaves {}, Python usa **indentación** (espacios o tabuladores) para estructurar el código.
- Esto mejora la **claridad**, la **organización** y asegura un estilo consistente.

Ejemplo 1: Mensaje simple

```
print("Hello, world!")
```

Ejemplo 2: Uso de condicionales con indentación

```
if grade == "A":  
    print("Excellent work!")  
else:
```

```
print("Keep studying and you'll improve!")
```

- Aquí la indentación define las acciones que debe ejecutar Python según la condición.
- Incluso sin conocer Python, la lógica se entiende por la estructura visual.

Ejemplo 3: Código sin indentación (incorrecto)

```
if grade == "A":  
  
print("Excellent work!")  
  
else:  
  
print("Keep studying and you'll improve!")
```

- Sin indentación, el código es desordenado, difícil de leer y Python generará un error.

En resumen: La **indentación en Python** no solo mejora la legibilidad, sino que es esencial para que el código funcione. Este diseño hace que Python sea más claro, intuitivo y accesible tanto para principiantes como para profesionales.

Tipos de datos en Python: los bloques de construcción del código

Los **tipos de datos** son las herramientas básicas para manejar información en un programa. Elegir el tipo correcto es clave para escribir código efectivo.

Principales tipos de datos:

1. Números

- Usados para representar cantidades.
- Ejemplo: **25** (edad), **19.99** (precio).

- Enteros (**int**) para números sin decimales, flotantes (**float**) para decimales.

2. Cadenas (**str**)

- Sirven para manejar texto.
- Ejemplo: "Alicia", "Guía del autoestopista galáctico", "¡Bienvenido!".

3. Booleanos (**bool**)

- Representan valores lógicos: Verdadero (**True**) o Falso (**False**).
- Ejemplo: verificar si una respuesta es correcta.

4. Listas (**list**)

- Colecciones ordenadas de valores.
- Ejemplo: ["leche", "huevos", "pan"].
- Útiles para almacenar múltiples elementos como puntuaciones o nombres.

5. Diccionarios (**dict**)

- Estructuras que guardan pares clave → valor.
- Ejemplo: {"nombre": "Alicia", "edad": 30, "email": "alicia@mail.com"}.
- Perfectos para representar información estructurada (como un perfil de usuario).

Los puntos fuertes de Python para principiantes

Python es considerado el **mejor punto de partida para aprender programación** gracias a una combinación de características que lo hacen accesible y poderoso:

1. Simplicidad y legibilidad:

Su sintaxis clara y similar al inglés permite a los principiantes concentrarse en la lógica en lugar de en complicaciones técnicas.

2. Comunidad de apoyo:

Existe una gran cantidad de tutoriales, foros y recursos, lo que asegura que los

estudiantes siempre encuentren ayuda e inspiración.

3. **Versatilidad:**

Python no se limita a un solo campo. Se utiliza en **desarrollo web, análisis de datos, automatización, inteligencia artificial y aprendizaje automático**, ofreciendo a los principiantes la posibilidad de explorar distintos ámbitos.

4. **Bibliotecas y frameworks:**

Su ecosistema rico en herramientas prediseñadas permite realizar tareas complejas con poco esfuerzo, como crear aplicaciones web o visualizaciones de datos.

5. **Integración con IA generativa:**

- **Generación de código:** asistentes como GitHub Copilot sugieren fragmentos útiles.
- **Depuración:** herramientas inteligentes detectan errores y ofrecen soluciones.
- **Aprendizaje personalizado:** la IA actúa como tutor virtual que adapta los materiales al nivel del estudiante.
- **Inspiración creativa:** propone enfoques alternativos para resolver problemas.

6. **Comparación con otros lenguajes:**

Aunque C++ o Java tienen ventajas de rendimiento, su complejidad y curva de aprendizaje son más pronunciadas. Python logra un equilibrio entre **potencia y sencillez**, lo que lo hace ideal para empezar.

En conclusión: Python no es solo un lenguaje de programación, sino una **puerta de entrada al mundo tecnológico**. Facilita el aprendizaje, potencia la creatividad y acompaña a los principiantes en su camino hacia convertirse en programadores seguros y competentes.

Python: la navaja suiza de la programación

Python es altamente adaptable y se utiliza en múltiples campos gracias a su **facilidad de uso, amplia colección de bibliotecas, comunidad activa y compatibilidad multiplataforma**.

Usos principales de Python en el mundo real:

1. Desarrollo web

- **Django**: framework completo para aplicaciones grandes (ej. Instagram, Spotify).
- **Flask**: micromarco flexible para proyectos pequeños y personalizados.

2. Ciencia de datos

- **Pandas**: manipulación y análisis de datos.
- **NumPy**: cálculos numéricos y trabajo con matrices.
- **Matplotlib**: visualización de datos mediante gráficos.

3. Automatización

- Scripts para organizar archivos o extraer información.
- **BeautifulSoup y Scrapy**: web scraping.
- Automatización de correos electrónicos.

4. Aprendizaje automático e IA

- **Scikit-learn**: clasificación, regresión y predicciones.
- **Keras y Microsoft Cognitive Toolkit**: redes neuronales y aprendizaje profundo (reconocimiento de imágenes, lenguaje natural, vehículos autónomos).

5. Otros campos

- **Pygame**: desarrollo de juegos en 2D.
- **SciPy**: investigación y simulaciones científicas.
- **Educación**: ideal como primer lenguaje por su simplicidad.

□ Ventajas de Python:

1. Sintaxis clara, fácil de leer y aprender.
2. Extenso ecosistema de bibliotecas y frameworks.
3. Comunidad activa con gran cantidad de recursos.

4. Compatibilidad multiplataforma (Windows, MacOS, Linux).

Cuestionario 1

1. Estás dirigiendo un taller de introducción a la programación para adultos sin experiencia previa en programación. Quieres elegir un lenguaje que les resulte accesible y fácil de aprender, aunque tengan una agenda muy apretada y diversos estilos de aprendizaje. ¿Por qué Python es una buena opción para estos alumnos? Seleccione la mejor respuesta.

- ☐ El amplio uso de bibliotecas de Python permite a los participantes crear rápidamente proyectos complejos, lo que les da una sensación de logro.
- ☒ La sintaxis sencilla de Python y su legibilidad facilitan la comprensión de los conceptos de programación, incluso con poco tiempo.
- ☐ El fuerte énfasis de Python en la programación orientada a objetos permite a los participantes explorar conceptos avanzados desde el principio.
- ☐ El amplio uso de Python en el desarrollo web permite a los participantes crear sitios web y aplicaciones interactivas, lo que aumenta sus perspectivas laborales.

✓ **Correcto**

Correcto La claridad de la sintaxis de Python es una gran ventaja para los alumnos que pueden tener una formación diversa.

2. Estás usando Python para escribir un programa que simule una simple máquina expendedora. El programa necesita: 1) Mostrar los artículos disponibles y sus precios, 2) Aceptar la selección del usuario y el pago, 3) Calcular el cambio, y 4) Dispensar el artículo seleccionado. ¿Cuál de las siguientes opciones muestra la mejor manera de abordar esta tarea utilizando los principios de descomposición de problemas en programación? Seleccione la mejor respuesta.

- ☒ Defina funciones separadas para cada paso: mostrar artículos, procesar el pago, calcular el cambio y dispensar el artículo.
- ☐ Priorice la optimización y la eficiencia del código desde el principio, empleando algoritmos avanzados para tareas como el cálculo de cambios.
- ☐ Desarrollar una estructura de clases compleja con herencia y polimorfismo para representar la máquina expendedora y sus interacciones.
- ☐ Escriba todo el programa como un bloque continuo de código, manejando todos los aspectos de la simulación de la máquina expendedora a la vez.

✓ **Correcto**

Correcto Descomponer el programa en funciones lo hace más organizado, legible y fácil de depurar.

3. Una empresa quiere renovar su plataforma de venta en línea para gestionar el aumento del tráfico de usuarios y ofrecer recomendaciones de productos personalizadas. Sus desarrolladores trabajan actualmente con una base de código heredada que es difícil de mantener y actualizar. ¿Cuál de las siguientes aplicaciones de Python podría satisfacer mejor las necesidades de la empresa? Seleccione todas las que correspondan.

- ☒ Uso de las funciones de Análisis de datos de Python para generar recomendaciones de productos personalizadas basadas en el historial de navegación y los patrones de compra de los usuarios

☒ **Correcto**

Correcto Las bibliotecas de análisis de datos de Python pueden utilizarse para analizar los datos de los usuarios y ofrecer recomendaciones de productos personalizadas, mejorando la experiencia del cliente.

- ☒ Implementación de las capacidades de Aprendizaje automático de Python para predecir futuras tendencias de ventas y optimizar la gestión de inventarios

☒ **Esto no debería estar seleccionado**

No del todo. Aunque Python es excelente para el aprendizaje automático, esto no está directamente relacionado con las necesidades inmediatas de la empresa de mejorar su plataforma de venta en línea.

- ☒ Utilización del marco Django de Python para una infraestructura web escalable que gestione el aumento del tráfico de usuarios

☒ **Correcto**

¡Correcto! Django es un potente framework de Python que se utiliza para gestionar millones de interacciones y POST de usuarios cada día, por lo que es ideal para sitios web con mucho tráfico.

- ☐ Uso de Python para automatizar el proceso de envío de notificaciones por correo electrónico sobre el lanzamiento de nuevos productos u ofertas especiales

4. Un equipo de desarrollo de software está empezando un nuevo proyecto que requiere un lenguaje de programación conocido por su legibilidad, facilidad de uso y capacidad para manejar tareas complejas. El equipo necesita un lenguaje que fomente una colaboración eficaz y permita un desarrollo y una modificación rápidos. ¿Cuál de las siguientes características de Python lo convertiría en una opción adecuada para este proyecto? Seleccione todas las que procedan.

- ☒ La naturaleza interpretada de Python permite una ejecución inmediata del código y una depuración más rápida, lo que facilita un desarrollo rápido.

✔ **Correcto**

Correcto La naturaleza interpretada de Python permite a los desarrolladores ejecutar el código línea por línea y ver los resultados al instante, lo que acelera el desarrollo y la depuración.

- ☒ La abstracción de alto nivel de Python simplifica las complejas interacciones de bajo nivel, lo que permite al equipo centrarse en la lógica del proyecto.

✔ **Correcto**

Correcto Python maneja muchas complejidades de bajo nivel, lo que hace que el código sea más limpio y fácil de entender, lo que es beneficioso para la colaboración y el mantenimiento.

- ☒ El paradigma de Programación orientada a objetos (POO) de Python fomenta la reutilización y modularidad del código, lo que permite un trabajo en equipo eficaz y unos procesos de desarrollo más ágiles.

✔ **Correcto**

Correcto Las funciones de programación orientada a objetos de Python, como las clases y los objetos, mejoran la organización y reutilización del código, lo que es crucial para los grandes proyectos y la colaboración en equipo.

- ☐ La estricta adhesión de Python a un único paradigma de codificación garantiza la coherencia en todos los proyectos y simplifica la integración con otras tecnologías.

5. Se te ha encargado crear un programa para almacenar la información de los alumnos, incluyendo sus nombres, edades y calificaciones. ¿Qué tipo de datos sería el más adecuado para organizar y acceder eficientemente a estos datos? Seleccione la mejor respuesta.

- ☐ Cuerdas
- ☒ Diccionarios
- ☐ Booleanos
- ☐ Listas

✔ **Correcto**

Correcto Los diccionarios son excelentes para almacenar datos en pares clave-valor. Puedes utilizar claves como "nombre", "edad" y "curso" para almacenar la información correspondiente a cada alumno, facilitando así su acceso y gestión.

6. Imagina que estás desarrollando un programa para analizar un gran conjunto de datos de opiniones de clientes para identificar tendencias y sentimientos. ¿Qué biblioteca de Python sería la más adecuada para manejar y manipular eficientemente estos datos textuales? Seleccione la mejor respuesta.

- ☐ Flask
- ☐ Pygame
- ☐ Scikit-learn
- ☒ pandas

✔ **Correcto**

Pandas es una biblioteca versátil de análisis de datos que proporciona herramientas para trabajar con datos estructurados, incluido el texto. Permite limpiar, transformar y analizar de forma eficiente grandes conjuntos de datos, lo que la convierte en una gran opción para gestionar las opiniones de los clientes.

IDE en Python

Un **IDE (Entorno de Desarrollo Integrado)** es una aplicación que reúne herramientas esenciales para programar de forma más eficiente.

Funciones principales de un IDE

- **Edición:** espacio para escribir y dar formato al código.
- **Depuración:** localizar y corregir errores paso a paso.

- **Autocompletado:** sugerencias de funciones y variables para ahorrar tiempo y reducir errores.
- **Control de versiones:** seguimiento de cambios e integración con Git.

Aunque se puede usar un editor de texto simple, un IDE ofrece más potencia, organización y productividad.

Ventajas de usar un IDE

1. Eficiencia:

- Autocompletado de código.
- Resaltado de sintaxis por colores.
- Formato automático para mantener el código limpio.

2. Depuración:

- Herramientas para rastrear errores y ejecutar el código paso a paso.

3. Organización:

- Facilita trabajar con proyectos que incluyen varios archivos, librerías y dependencias.

4. Colaboración:

- Integración con control de versiones y funciones de trabajo en equipo.

IDE más populares para Python

● **Jupyter Notebook**

- Ideal para ciencia de datos e investigación.
- Celdas interactivas para ejecutar código por partes.

- Visualización integrada de gráficos y resultados.
- Soporte para Markdown (texto + ecuaciones + imágenes).
- **VS Code**
 - Personalizable con muchas extensiones.
 - Finalización inteligente de código.
 - Depuración con puntos de interrupción.
 - Integración con Git.
 - Marketplace con extensiones para distintos campos (web, datos, etc.).
- **Otros IDE:**
 - **PyCharm:** profesional, con herramientas avanzadas de depuración, refactorización y pruebas.
 - **VS Code con extensión de Python:** soporte completo para codificación, depuración y terminal integrado.

Conclusión

Un IDE no es solo un editor de texto: es una herramienta que aumenta la **eficiencia, productividad, organización y colaboración** en proyectos de Python. La elección del IDE depende del nivel del programador y del tipo de proyectos que vaya a realizar.

Jupyter Notebook

Qué es

- Una aplicación web que permite crear y compartir documentos que combinan **código en vivo, resultados, texto, visualizaciones y ecuaciones**.
- Funciona con **celdas** que pueden contener código o texto, lo que fomenta el aprendizaje interactivo y la documentación clara.

- Los archivos tienen extensión **.ipynb**.

Instalación

- **Con Anaconda:** ya viene incluido y aporta herramientas de análisis y ciencia de datos.
- **Con pip:** ejecutar `pip install notebook`. Requiere tener Python previamente instalado.
- Para iniciar: `jupyter notebook` → abre el **dashboard** en el navegador.

Ventajas principales

1. **Aprendizaje interactivo:** ejecución por celdas con resultados inmediatos.
2. **Visualización de datos:** integración con bibliotecas como Matplotlib para gráficos y análisis visual.
3. **Documentación y uso compartido:** permite añadir texto con Markdown, imágenes y explicaciones.
4. **Experimentación y prototipado:** ideal para probar ideas, algoritmos o modelos de manera ágil.
5. **Colaboración:** se pueden compartir notebooks en múltiples formatos (HTML, PDF, Python script) y usar plataformas como JupyterHub o Azure.

Aplicaciones en la práctica

- **Ciencia de datos:** análisis exploratorio, predicciones y visualización de resultados.
- **Aprendizaje automático:** experimentación con algoritmos y mejora iterativa de modelos.

- **Educación:** creación de lecciones interactivas con ejercicios.
- **Investigación científica:** análisis de datos complejos (ej. cambio climático).
- **Modelización financiera:** simulaciones, análisis de riesgos y valoración de activos.

Características avanzadas

- **Comandos mágicos:** atajos como `%timeit` para medir tiempos de ejecución.
- **Extensiones:** mejoras de la interfaz, temas oscuros o soporte para otros lenguajes (R, Julia).
- **Widgets:** elementos interactivos como sliders, menús o botones para enriquecer la exploración de datos.

Limitaciones

- El trabajo con muchas celdas puede producir código fragmentado y difícil de mantener.
- Cuadernos grandes pueden desorganizarse.
- Se recomienda usar **funciones**, buenas prácticas de nomenclatura y dividir proyectos en varios notebooks.

Conclusión

Jupyter Notebook es una herramienta **indispensable para programadores de Python**, especialmente en **ciencia de datos, machine learning e investigación**. Combina interactividad, visualización, documentación y colaboración, convirtiéndose en un entorno único para **aprender, experimentar y compartir conocimientos**.

Cuestionario 1

1. Un usuario está configurando su entorno de desarrollo de Python y desea garantizar un proceso de instalación sin problemas con acceso a una amplia gama de paquetes preinstalados de uso común en la Ciencia de datos y el Aprendizaje automático. También desea poder gestionar fácilmente distintos entornos de proyecto y evitar conflictos entre paquetes. ¿Cuál de las siguientes herramientas satisfaría mejor sus necesidades? Seleccione la mejor respuesta.

- ☐ Una instalación estándar de Python con gestión manual de paquetes
- ☒ Una instalación de Python con la distribución Anaconda
- ☐ Una instalación de Python utilizando un entorno virtual y la Biblioteca estándar de Python
- ☐ Una instalación de Python utilizando una distribución mínima y pip para la instalación de paquetes

✓ **Correcto**

Correcto Anaconda ofrece un completo sistema de gestión de paquetes con muchas bibliotecas de ciencia de datos preinstaladas y la herramienta `conda` para facilitar la gestión del entorno.

2. Un científico de datos está trabajando en un proyecto que requiere versiones específicas de varios paquetes de Python. Necesita crear un entorno aislado para evitar conflictos con otros proyectos y garantizar la reproducibilidad. ¿Cuál de las siguientes herramientas sería más eficaz para gestionar este escenario? Seleccione la mejor respuesta.

- ☐ Cuadernos Jupyter
- ☐ Pip
- ☒ Conda
- ☐ Biblioteca estándar de Python

✓ **Correcto**

¡Correcto! Conda es un potente administrador de paquetes y entornos que permite crear entornos aislados con versiones de paquetes específicas.

3. Un científico de datos está comenzando un nuevo proyecto que implica explorar un gran conjunto de datos, crear visualizaciones y compartir sus hallazgos con colegas en un formato fácilmente comprensible. Necesitan un IDE que facilite la codificación interactiva, admita resultados ricos como tablas y gráficos, y permita la integración de código con texto explicativo y visualizaciones. ¿Cuál de los siguientes IDE se adapta mejor a sus necesidades? Seleccione la mejor respuesta.

- ☐ VS Code con extensión Python
- ☒ Notebook de Jupyter
- ☐ Un editor de texto básico con un intérprete de Python
- ☐ PyCharm

✓ **Correcto**

¡Correcto! Notebook de Jupyter es ideal para la exploración interactiva de datos, la visualización y el intercambio de resultados gracias a su compatibilidad con celdas interactivas, resultados enriquecidos e integración con Markdown.

4. Un equipo de científicos de datos está colaborando en un proyecto de Aprendizaje automático. Necesitan una herramienta que les permita compartir fácilmente código, visualizaciones y texto explicativo en un único documento interactivo. También quieren poder experimentar con diferentes fragmentos de código y ver los resultados inmediatamente. ¿Cuál de las siguientes herramientas satisfaría mejor las necesidades de este equipo de Ciencia de datos? Seleccione la mejor respuesta.

- ☐ Un intérprete estándar de Python en una ventana de terminal
- ☒ Notebook de Jupyter
- ☐ Un programa de hojas de cálculo como Microsoft Excel
- ☐ Un editor de texto básico con resaltado de sintaxis

✓ **Correcto**

¡Correcto! Notebook de Jupyter es ideal para proyectos colaborativos de ciencia de datos debido a su capacidad para combinar código, visualizaciones y texto en un entorno interactivo.

5. Un científico de datos está explorando un nuevo conjunto de datos y quiere experimentar con diferentes transformaciones y visualizaciones de datos. Necesitan una herramienta que les permita ejecutar código en pequeños fragmentos, ver los resultados inmediatamente y modificar fácilmente el código para iterar en su análisis. ¿Qué característica de Notebook de Jupyter lo hace especialmente adecuado para este tipo de exploración interactiva de datos? Seleccione la mejor respuesta.

- ☒ El soporte de celdas markdown para documentar el código y el análisis.
- ☐ Capacidad para crear presentaciones con código incrustado y visualizaciones.
- ☐ La capacidad de organizar el código en celdas que pueden ejecutarse de forma independiente.
- ☐ La integración con sistemas de control de versiones como Git.

✗ **Incorrecto**

No del todo. La documentación Markdown es útil para la claridad y la comunicación, pero la naturaleza interactiva de Notebook de Jupyter es principalmente posible gracias a su estructura basada en celdas.

Introducción a Python y el programa "¡Hola, mundo!"

1. Significado de "¡Hola, mundo!"

- Es el primer programa tradicional para quienes aprenden a programar.
- Su función principal es introducir la sintaxis básica de Python y generar confianza en los principiantes.

- Permite verificar que el entorno Python está correctamente instalado y funcionando.

2. Estructura del programa

Código básico:

```
print("Hello, World!")
```

-
- `print()` muestra mensajes en pantalla, y las comillas delimitan texto.
- Este sencillo programa enseña los fundamentos de Python: sentencias, funciones y literales de cadena.

3. Entornos de desarrollo recomendados

- **Jupyter Notebook / JupyterLab:** Ideal para análisis de datos y aprendizaje interactivo.
- **Visual Studio Code:** Editor flexible y extensible para múltiples proyectos.
- **PyCharm:** IDE profesional con depuración avanzada y control de versiones.
- **Thonny:** IDE simple y visual para principiantes.

4. Cómo funciona Python

- **Lectura:** Python analiza el código línea por línea y lo convierte en tokens.
- **Interpretación:** Comprende la intención del código según la sintaxis de Python.
- **Ejecución:** Traduce a bytecode y la PVM lo ejecuta para mostrar el resultado en pantalla.

5. Importancia de aprender Python

- “¡Hola, mundo!” simboliza el inicio de la programación y la capacidad de dar órdenes a la computadora.

- Python es versátil: desarrollo web, análisis de datos, automatización, IA, juegos y más.
- Permite construir programas complejos y resolver problemas del mundo real.

6. Próximos pasos

- Aprender los fundamentos y construir sobre ellos.
- Explorar proyectos más avanzados según tus intereses: datos, web, IA, automatización.
- Integrarse en la comunidad global de desarrolladores de Python.

Sintaxis y estructura en Python

1. Sintaxis de Python

- La sintaxis es el conjunto de reglas que permite comunicar tus ideas al ordenador mediante código.
- Cuatro elementos fundamentales: **indentación, sentencias, variables y comentarios**.

2. Elementos clave

- **Indentación:** Python usa espacios al inicio de cada línea para definir bloques de código. Cada nivel indica jerarquía y facilita la lectura y el mantenimiento.
- **Sentencias:** Son instrucciones que realizan acciones específicas en el programa, como cálculos, asignaciones o llamadas a funciones.
- **Variables:** Contenedores para almacenar y manipular datos (números, texto, listas, etc.), haciendo el código flexible y reutilizable.
- **Comentarios:** Texto que explica el código; Python los ignora durante la ejecución. Se usan para clarificar la lógica y documentar el programa.

3. Programa “Hello World” en Python

Pasos para crear y ejecutar:

1. Crear un archivo `.py` (ej. `hello.py`).

Escribir el código:

```
print("Hello, World!")
```

- 2.
 3. Guardar y ejecutar el archivo en el IDE o mediante terminal (`python hello.py`).
- Esto imprime “Hello World” en pantalla y sirve como primera práctica de sintaxis.

4. Concepto general

- Dominar la sintaxis es como aprender a “hablar” con la computadora.
- La práctica constante ayuda a comprender mejor la estructura y a escribir código claro, eficiente y funcional.

Operaciones aritméticas y variables en Python

1. Importancia de Python para los cálculos

- Python facilita el cálculo rápido, preciso y automatizado de datos, reduciendo errores humanos.
- Es flexible y puede manejar desde operaciones básicas hasta cálculos complejos, análisis estadístico y matemáticas avanzadas.

2. Operaciones aritméticas básicas

- **Suma (+):** Combina valores, calcula totales.
- **Resta (-):** Encuentra diferencias, márgenes o saldos.
- **Multiplicación (*):** Calcula áreas, totales o cantidades repetidas.
- **División (/):** Encuentra proporciones o precios por unidad.

3. Variables en Python

- Actúan como contenedores para almacenar, recuperar y manipular datos.
- Permiten usar nombres descriptivos, hacer el código más legible y manejar información dinámica (por ejemplo, entradas de usuario).
- Facilitan la reutilización de datos y la creación de programas más flexibles y sofisticados.

4. Aplicaciones prácticas

- Automatización de cálculos repetitivos.
- Análisis financiero, científico o de datos.
- Base para programas simples y aplicaciones complejas.

Variables en Python

1. Qué son las variables

- Contenedores que almacenan datos con nombres significativos.
- Funcionan como “notas adhesivas virtuales” para organizar y acceder a información dentro del programa.

2. Beneficios principales

- **Legibilidad:** Nombres descriptivos reemplazan valores crípticos, facilitando la comprensión del código.
- **Reutilización:** Permiten usar un valor en múltiples lugares sin repetirlo, simplificando el mantenimiento.
- **Flexibilidad:** Los valores pueden cambiar dinámicamente según la entrada del usuario, cálculos o condiciones externas.

3. Cómo crear y usar variables

Se asigna un valor mediante el signo igual (=), por ejemplo:

```
nombre_empleado = "Alice"
```

-
- Se pueden reasignar valores para reflejar cambios, como actualizar la puntuación en un juego o la temperatura en una app meteorológica.
- Nombres claros y comentarios mejoran la colaboración y comprensión del código.

4. Aplicaciones prácticas

- **Comercio electrónico:** Guardar nombre del cliente, productos, costos y generar pedidos personalizados.
- **Análisis de datos:** Organizar y manipular información sobre tendencias, métricas y resultados científicos.
- **Automatización de tareas:** Scripts que usan variables dinámicas para informes, notificaciones o gestión de sistemas.

5. Conclusión

- Las variables son esenciales para escribir código limpio, flexible y colaborativo.
- Practicar su uso permite dominar la manipulación de datos y desarrollar aplicaciones más complejas en Python.

Ejecutar programas Python y comprensión de Jupyter

1. Primeros pasos en Python

- Aprender a programar es como una aventura: todo comienza con ejecutar tu primer programa, típicamente "¡Hola, mundo!".
- Aunque se usará principalmente **Jupyter Notebooks**, entender cómo funciona el **intérprete de Python** y la **línea de comandos** es fundamental.

2. Cómo funciona la ejecución de código

- El código Python es texto con instrucciones que el **intérprete** traduce y ejecuta línea por línea.
- La **línea de comandos** permite enviar el código al intérprete, ejecutar scripts (`python archivo.py`) y gestionar entornos, paquetes y argumentos.

3. Ventajas de comprender la línea de comandos

- Flexibilidad: funciona igual en distintos sistemas operativos.
- Depuración: ayuda a identificar errores leyendo la salida del intérprete.
- Automatización: permite ejecutar tareas repetitivas y scripts de forma eficiente.

4. Relación con Jupyter Notebooks

- Jupyter simplifica la escritura y ejecución de código, integrando texto, visualizaciones y resultados en un solo lugar.
- Internamente, Jupyter envía el código al intérprete de Python, por lo que entender el proceso subyacente mejora la resolución de problemas y la gestión del entorno.

5. IDEs y herramientas alternativas

- Herramientas como VS Code, PyCharm o Jupyter facilitan la programación con depuración, autocompletado y gestión de proyectos.
- La elección del entorno depende de la preferencia personal y del tipo de proyecto.

6. Conclusión

- Ejecutar tu primer programa es un hito importante.
- Comprender cómo Python interpreta el código y cómo funciona la línea de comandos proporciona una base sólida para proyectos más complejos y para usar Jupyter de manera más efectiva

Sentencias y expresiones en Python

1. Sentencias: Instrucciones que dirigen el programa

- Son líneas de código que indican al ordenador qué hacer.
- **Tipos principales:**
 - **Asignación:** `age = 25` guarda un valor en una variable.

Condicionales (if-else): permiten tomar decisiones según condiciones evaluadas con operadores (`==`, `!=`, `>`, `<`, `>=`). Ejemplo:

```
age = 25
```

```
if age >= 18:
```

```
    print("You are eligible to vote.")
```

```
else:
```

```
    print("You are not yet eligible to vote.") #puede votar
```

-
- **Impresión:** `print()` muestra resultados o información al usuario.
- **Bucles:** `for` y `while` repiten tareas automáticamente.
- **Funciones:** Agrupan instrucciones reutilizables (`def nombre_funcion():`).
- **Clases:** Base de la programación orientada a objetos; crean objetos con atributos y métodos.

2. Expresiones: Cálculos que producen valores

- Realizan operaciones dentro de las sentencias para generar resultados.
- **Operaciones aritméticas básicas:** `+`, `-`, `*`, `/`
- **Operaciones adicionales:**

- `//` División entera
- `%` Módulo (resto de división)
- `**` Exponenciación

Las expresiones pueden combinarse y almacenarse en variables para cálculos complejos, como descuentos o áreas:

```
discount = 0.2  
  
price = 50  
  
final_price = price * (1 - discount)  
  
print(final_price)  # 40.0
```

•

3. Conclusión

- **Sentencias** dictan el flujo y las acciones del programa.
- **Expresiones** calculan valores y manipulan datos para obtener resultados significativos.
- Juntas, permiten construir programas eficientes, adaptables y con lógica compleja.

Automatización en Python y cálculo de precios

1. Automatización en Python

- Python permite **automatizar tareas repetitivas** y complejas, aumentando eficiencia, precisión y reproducibilidad.
- **Automatización básica:**
 - **Iterar:** recorrer archivos o datos con bucles (`for`) para procesar colecciones completas.
 - **Extraer:** leer y filtrar datos relevantes usando bibliotecas como `pandas` o `csv`.

```
import csv

with open('data.csv', mode='r') as file:

    csv_reader = csv.reader(file)

    for row in csv_reader:

        print(row)
```

-
- **Automatización avanzada:** uso de IA, aprendizaje automático, análisis estadístico, visualización (Matplotlib) e informes automatizados.
- Otras aplicaciones incluyen **web scraping**, **automatización de correos electrónicos** y **automatización de pruebas de software**.

2. Beneficios de la automatización

- Ahorra tiempo, reduce errores y mejora la eficiencia.
- Permite analizar grandes volúmenes de datos y tomar decisiones informadas rápidamente.

3. Ejemplo práctico: Calculadora de precios con descuento

- Tarea: calcular el precio final tras aplicar un descuento.

```
original_price = 75

discount_rate = 15


discount = (discount_rate / 100) * original_price

final_price = original_price - discount


print("The final price after discount is: $", final_price)
```

- **Resultado esperado:** The final price after discount is: \$ 63.75
- Demuestra cómo Python combina variables, expresiones y operaciones aritméticas para automatizar cálculos cotidianos.

4. Conclusión

- La automatización con Python simplifica tareas repetitivas y complejas.
- Con práctica, permite resolver problemas, crear aplicaciones eficientes y obtener resultados precisos rápidamente.

Cuestionario 3

1. Tu tarea es crear un programa en Python que simule un simple chatbot. Mientras escribes el código, quieres asegurarte de que es fácilmente comprensible tanto para ti como para tus compañeros de equipo. ¿Cuál de los siguientes elementos de la sintaxis de Python sería más crucial para organizar el flujo de diálogo del chatbot y hacer que el código sea legible? Selecciona la mejor respuesta.

- ☒ Indentación
- ☐ Comentarios
- ☐ Declaraciones
- ☐ Variables

✓ **Correcto**

Correcto La sangría juega un papel clave en la estructuración visual de los bloques de código, facilitando el seguimiento del flujo de conversaciones y respuestas en tu programa chatbot.

2. Estás trabajando en un programa que necesita calcular el número de semanas que han pasado desde el comienzo del año. Tienes el número total de días transcurridos. ¿Qué operación de Python sería la más adecuada para este cálculo? Selecciona la mejor respuesta.

- ☐ Resta
- ☐ Exponentes
- ☒ División
- ☐ Adición

✓ **Correcto**

Correcto La división te daría el número de semanas.

3. Estás desarrollando un programa para seguir el progreso de una campaña de marketing. Necesitas almacenar el número de visitas al sitio web, el número de clientes potenciales generados y el número de conversiones. ¿Cómo crearías y asignarías correctamente estos valores a variables en Python? Selecciona la mejor respuesta.

- ☐ `website_visits, leads_generated, conversions = 1500, 50, 10`
- ☒ `website_visits = 1500`
`leads_generated = 50`
`conversions = 10`
- ☐ `website_visits = 1500,`
`leads_generated = 50,`
`conversions = 10`
- ☐ `int website_visits = 1500;`
`int leads_generated = 50;`
`int conversions = 10;`

✔ **Correcto**

¡Correcto! Esto demuestra la forma correcta de crear y asignar valores a variables en Python, con cada variable en una nueva línea.

4. Estás trabajando en un proyecto con un equipo de otros desarrolladores y necesitas compartir tu programa "¡Hola, mundo!" con ellos. Decides utilizar un editor de código para escribir y guardar tu programa. ¿Cuáles de las siguientes características encontrarías en un editor de código o IDE típico mientras trabajas en tu programa "¡Hola, mundo! Selecciona todas las que correspondan.

- ☐ Un pincel para dibujar imágenes dentro del código.
- ☐ Una calculadora para realizar cálculos matemáticos.
- ☒ Un panel editor de código para escribir y editar el código.

✔ **Correcto**

Correcto El panel del editor de código es una característica fundamental donde se escribe y modifica el código.

- ☒ Un botón "Ejecutar" para ejecutar el código.

✔ **Correcto**

Correcto Los editores de código y los IDE tienen un botón "Ejecutar" o un atajo de teclado para ejecutar el código.

5. Tienes un script de Python llamado `my_script.py` que contiene la línea `print('Hello, World!')`. ¿Cuál de las siguientes afirmaciones describe con precisión las diferencias clave entre ejecutar este script desde la línea de comandos y ejecutarlo dentro de un Notebook de Jupyter?

- ☐ Ejecutar `python my_script.py` en la línea de comandos permite modificar fácilmente el código del script directamente en el terminal, mientras que Notebook de Jupyter requiere utilizar un editor de texto externo.
- ☐ Tanto la línea de comandos como el Notebook de Jupyter ejecutan el script de la misma forma, sin diferencias significativas en cómo se procesa el código o cómo se interactúa con él.
- ☐ La línea de comandos es más interactiva que Notebook de Jupyter, lo que permite modificar y ejecutar el código en tiempo real.
- ☒ La ejecución de `python my_script.py` en la línea de comandos ejecuta el script como un archivo independiente, mientras que el Notebook de Jupyter permite ejecutar celdas de código de forma interactiva y examinar variables entre ejecuciones

✔ **Correcto**

Correcto Esto capta con precisión la diferencia fundamental. La línea de comandos ejecuta el script de principio a fin. Notebook de Jupyter permite la ejecución interactiva, celda por celda, lo que es genial para la exploración y depuración.

6. Un programador está creando un programa para automatizar una tarea repetitiva. Quiere ver los resultados de su código a medida que el programa se ejecuta. ¿Cómo suelen mostrar la mayoría de los lenguajes de programación, incluido Python, los resultados de la ejecución del código? Seleccione la mejor respuesta.

- ☐ La salida se guarda directamente en un archivo, por lo que el programador debe abrirlo para ver los resultados.
- ☐ La salida está incrustada dentro del propio código, apareciendo como comentarios junto a las líneas pertinentes.
- ☒ La salida se muestra en una ventana dedicada dentro del entorno de codificación, a menudo llamada consola o terminal.
- ☐ La salida se imprime automáticamente en una impresora física conectada al ordenador.

✔ **Correcto**

Correcto La consola o terminal es el lugar típico para ver la salida basada en texto del código.

7. Se le ha encargado la creación de un programa para analizar los datos de ventas de una gran empresa minorista. Esto implica procesar miles de registros de ventas para identificar tendencias, calcular los ingresos totales y generar informes. ¿Cuáles de los siguientes son ejemplos de cómo Python podría utilizarse para automatizar este Proceso de análisis de datos? Seleccione todo lo que corresponda.

- ☐ Python podría utilizarse para crear presentaciones visualmente atractivas con animaciones y transiciones para mostrar los resultados del análisis a los interesados.
- ☒ Python podría utilizarse para realizar cálculos sobre los datos de ventas, como calcular los ingresos totales, identificar los productos más vendidos y calcular el Valor promedio de pedido.

✔ **Correcto**

Correcto Python destaca en la realización de operaciones matemáticas y estadísticas con datos, por lo que resulta ideal para tareas como el cálculo de ingresos y la identificación de tendencias.

- ☒ Python podría utilizarse para generar informes que resuman los resultados del análisis, incluidos cuadros, gráficos e indicadores clave de rendimiento.

✔ **Correcto**

Correcto Python puede utilizarse para generar informes en varios formatos, como texto, hojas de cálculo y PDF, lo que facilita compartir los resultados del análisis.

- ☒ Python podría utilizarse para extraer automáticamente datos relevantes de los registros de ventas, como ID de productos, cantidades vendidas y precios.

✔ **Correcto**

Correcto Python puede utilizarse para leer y filtrar datos de diversas fuentes, incluidas bases de datos y hojas de cálculo.

ACTIVIDAD 1

1. Eres un científico de datos que trabaja en un proyecto que implica el análisis de grandes conjuntos de datos y la creación de modelos predictivos. Necesitas limpiar, transformar y visualizar los datos antes de aplicar algoritmos de aprendizaje automático. ¿Cómo ayudan las bibliotecas de Python en este proceso? Selecciona la mejor respuesta.

- ☒ Bibliotecas como NumPy y Pandas proporcionan herramientas eficaces para la manipulación, el análisis y la visualización de datos, mientras que Scikit-learn ofrece una amplia gama de algoritmos de aprendizaje automático.
- ☐ Bibliotecas como Django y Flask proporcionan una base estructurada para construir aplicaciones web, permitiendo visualizaciones interactivas de datos.
- ☐ Las bibliotecas especializadas en la administración de la memoria de bajo nivel garantizan el procesamiento eficaz de grandes conjuntos de datos.
- ☐ Las bibliotecas centradas en la automatización y el scripting permiten agilizar los pasos de recogida y procesamiento de datos.

✓ **Correcto**

¡Correcto! Estas bibliotecas son esenciales para las tareas de Ciencia de datos en Python, proporcionando las herramientas necesarias para la manipulación de datos, análisis y aprendizaje automático.

2. Estás ayudando a un amigo a aprender Python. Está confundido sobre por qué su código no funciona. Te das cuenta de que han escrito lo siguiente:

```
name = "Elias"  
    print(name)
```

¿Cuál es la razón MÁS probable de un error en este código, basado en los principios de la sintaxis de Python? Selecciona la mejor respuesta.

- ☒ La función de impresión no debe tener sangría.
- ☐ La función `print` debe utilizar comillas simples en lugar de dobles.
- ☐ La variable `name` está declarada incorrectamente.
- ☐ Las variables no se pueden imprimir.

✓ **Correcto**

Correcto En Python, la sangría es crucial para definir bloques de código. La sentencia `print` no puede tener sangría.

3. Usted es un investigador que estudia los efectos del cambio climático en los patrones migratorios de las aves. Necesita analizar grandes conjuntos de datos de avistamientos de aves, patrones meteorológicos y datos geográficos. ¿Qué características de Python serían más útiles en este caso? Seleccione todas las que corresponda.

☒ Un rico ecosistema de bibliotecas

✔ **Correcto**

Correcto La amplia colección de bibliotecas de Python, incluidas las específicas para la computación científica como NumPy y Matplotlib, sería inestimable para su investigación.

☒ Procesamiento paralelo integrado

✘ **Esto no debería estar seleccionado**

No del todo. Consulta *Python in the wild: From web apps to machine learning* para más información.

☐ Tipado estático fuerte

☒ Capacidades informáticas interactivas

✔ **Correcto**

Correcto Herramientas como los Notebooks de Jupyter permiten explorar y visualizar datos de forma interactiva, lo que ayuda a comprender patrones climáticos y migratorios complejos.

4. Estás a punto de empezar un nuevo proyecto de Análisis de datos. Necesitas abrir el Notebook de Jupyter para empezar a escribir tu código. ¿Qué comando escribirás en tu terminal para iniciar Notebook de Jupyter? Selecciona la mejor respuesta.

- ☐ `run jupyter notebook`
- ☒ `jupyter notebook`
- ☐ `python -m jupyter`
- ☐ `start jupyter`

✓ **Correcto**

Correcto Este comando iniciará el servidor de Notebook de Jupyter y abrirá la interfaz en tu navegador web.

5. Estás comenzando un nuevo proyecto que requiere Python, y tu equipo ha decidido utilizar la distribución Anaconda. Eres responsable de configurar tu entorno de desarrollo. ¿Qué es lo primero que tienes que hacer para instalar Python en tu máquina Windows? Selecciona la mejor respuesta.

- ☐ Selección de los paquetes y bibliotecas deseados durante la instalación
- ☐ Configuración de variables de entorno para Anaconda
- ☒ Descarga del instalador de Anaconda desde el sitio web oficial
- ☐ Ejecutar el instalador con privilegios de administrador

✓ **Correcto**

Correcto El primer paso es descargar la distribución de Anaconda desde el sitio web oficial.

6. Eres un científico de datos trabajando en un proyecto que requiere que realices cálculos complejos, crees visualizaciones y compartas tus hallazgos con tus colegas en un formato fácilmente comprensible. Necesitas un IDE que te facilite estas tareas sin problemas. ¿Qué IDE se adapta mejor a tus necesidades? Seleccione la mejor respuesta.

- ☐ PyCharm
- ☐ Anaconda
- ☒ Notebook de Jupyter
- ☐ Código VS

✔ **Correcto**

¡Correcto! Notebook de Jupyter es ideal para la exploración interactiva de datos, la visualización y el intercambio de resultados gracias a su compatibilidad con celdas interactivas, resultados enriquecidos y markdown.

7. Eres nuevo en Python e intentas escribir tu primer programa, el clásico "¡Hola, mundo!", en Visual Studio Code. Has escrito el siguiente código:

```
print("Hello, world!")
```

Cuando intentas ejecutarlo, aparece un error. ¿Cuál es la razón más probable del error? Selecciona la mejor respuesta.

- ☐ No has definido una variable para almacenar la cadena "¡Hola, mundo!".
- ☐ Para imprimir texto en Python es necesario utilizar un comando especial.
- ☐ Olvidaste añadir un comentario explicando el código.
- ☒ No has guardado el archivo con extensión .py.

✔ **Correcto**

Correcto Es importante guardar los archivos Python con la extensión .py antes de ejecutarlos.

8. Usted está desarrollando un juego en el que un jugador gana puntos por completar diferentes tareas. Necesitas calcular la puntuación total del jugador sumando puntos por tareas, restando puntos por penalizaciones y multiplicando los puntos de bonificación. ¿Cuál de las siguientes expresiones de Python podría utilizarse en el sistema de puntuación de tu juego? Seleccione todas las que correspondan.

☒ `bonus_points = 2 ** bonus_level`

✔ **Correcto**

Correcto Utiliza el operador de exponenciación para calcular los puntos de bonificación en función del nivel de bonificación del jugador.

☐ `final_score = current_score / 0`

☒ `score = initial_points + task_points - penalty_points`

✔ **Correcto**

Correcto Combina la suma y la resta para calcular la puntuación total.

☒ `score = 0`

✔ **Correcto**

Correcto La variable `score` puede ponerse a cero antes de una partida, o inicializarse antes de su uso.

9. Imagina que estás explicando cómo Python ejecuta un programa. ¿Cuál de las siguientes opciones describe con precisión la secuencia de acciones que Python realiza, desde la lectura del código hasta la visualización de la salida? Selecciona la mejor respuesta.

- ☒ Lectura del código línea por línea, interpretación de su significado y ejecución de las instrucciones.
- ☐ Producir la salida y, a continuación, ejecutar las instrucciones.
- ☐ Interpretar el significado del código, ejecutar las instrucciones y, a continuación, leer el código línea por línea.
- ☐ Ejecutar las instrucciones, leer el código línea por línea e interpretar su significado.

✓ **Correcto**

Correcto Esto refleja el proceso de Python: análisis léxico (lectura), análisis sintáctico (interpretación) y, por último, ejecución.

10. Un equipo de programadores está desarrollando un programa para simular el comportamiento de un sistema complejo con componentes que interactúan entre sí. ¿Cuál de las siguientes características de Python sería más útil para representar estos componentes y permitir a los programadores trabajar de forma independiente? Seleccione la mejor respuesta.

- ☐ Declaraciones de bucle
- ☒ Definiciones de funciones
- ☐ Imprimir declaraciones
- ☐ Operaciones de archivo

✓ **Correcto**

Correcto Las definiciones de clase permiten crear objetos que pueden encapsular datos y comportamiento, lo que las hace ideales para representar entidades del mundo real y sus interacciones.